

Practical No: 08

Aim:

To perform data aggregation using MongoDB aggregation pipeline.

Theory:

Aggregation in MongoDB is a powerful framework used to process and transform data within a collection. Similar to SQL operations like GROUP BY, SUM(), COUNT(), and AVG(), the aggregation pipeline enables users to perform complex computations, data restructuring, and analytical queries.

The foundation of MongoDB's aggregation system is the **aggregation pipeline**, which processes documents through a sequence of stages. Each stage receives input from the previous one, performs an operation, and passes output to the next stage—similar to how water flows through connected pipes.

One of the most commonly used stages is **\$group**, which groups documents by a specified key and performs accumulator operations such as counting, summing, averaging, finding minimum/maximum values, and more. Besides \$group, MongoDB provides many other stages such as:

- **\$match** – filters documents (similar to SQL WHERE)
- **\$project** – reshapes or selects fields
- **\$sort** – sorts the processed output
- **\$lookup** – performs joins across collections
- **\$limit**, **\$skip** – pagination controls

In this experiment, using \$group helps summarize student data by department, producing meaningful insights from stored records. Such operations are extremely useful in real-world applications such as attendance tracking, sales reports, demographic analysis, and system monitoring.

Commands:

```
db.students.aggregate([ { $group: { _id: "$dept", count: { $sum: 1 } } } ])
```

Example Students Collection

```
{ "_id": 1, "name": "Rahul", "dept": "Computer", "marks": 85 }
```

```
{ "_id": 2, "name": "Sneha", "dept": "Computer", "marks": 90 }
```

```
{ "_id": 3, "name": "Amit", "dept": "IT", "marks": 70 }
```

```
{ "_id": 4, "name": "Priya", "dept": "IT", "marks": 95 }
```

```
{ "_id": 5, "name": "Kiran", "dept": "Mechanical", "marks": 60 }
```

1) \$match – Filter Documents (like SQL WHERE)

Example: Find students with marks greater than 80

```
db.students.aggregate([  
  { $match: { marks: { $gt: 80 } } }  
])
```

Output

```
{ "_id": 1, "name": "Rahul", "dept": "Computer", "marks": 85 }  
{ "_id": 2, "name": "Sneha", "dept": "Computer", "marks": 90 }  
{ "_id": 4, "name": "Priya", "dept": "IT", "marks": 95 }
```

2) \$project – Select or Reshape Fields

Example: Display only name and marks

```
db.students.aggregate([  
  {  
    $project: {  
      name: 1,  
      marks: 1,  
      _id: 0  
    }  
  }  
])
```

Output

```
{ "name": "Rahul", "marks": 85 }  
{ "name": "Sneha", "marks": 90 }
```

```
{ "name": "Amit", "marks": 70 }
```

3) **\$sort** – Sort Results

Example: **Sort students by marks in descending order**

```
db.students.aggregate([  
  { $sort: { marks: -1 } }  
])
```

Output

```
{ "name": "Priya", "marks": 95 }  
{ "name": "Sneha", "marks": 90 }  
{ "name": "Rahul", "marks": 85 }  
{ "name": "Amit", "marks": 70 }  
{ "name": "Kiran", "marks": 60 }
```

4) **\$lookup** – Join Two Collections

Assume another collection **departments**

```
{ "dept": "Computer", "hod": "Dr. Sharma" }  
{ "dept": "IT", "hod": "Dr. Patil" }  
{ "dept": "Mechanical", "hod": "Dr. Joshi" }
```

Example: **Join students with department details**

```
db.students.aggregate([  
  {  
    $lookup: {  
      from: "departments",  
      localField: "dept",
```

```
    foreignField: "dept",
    as: "dept_info"
  }
}
])
```

Output

```
{
  "name": "Rahul",
  "dept": "Computer",
  "dept_info": [{ "dept": "Computer", "hod": "Dr. Sharma" }]
}
```

5) **\$limit** – Limit Number of Results

Example: **Show only first 2 students**

```
db.students.aggregate([
  { $limit: 2 }
])
```

Output

```
{ "name": "Rahul", "marks": 85 }
{ "name": "Sneha", "marks": 90 }
```

6) **\$skip** – Skip Documents (Pagination)

Example: **Skip first 2 students**

```
db.students.aggregate([
  { $skip: 2 }
```

])

Output

```
{ "name": "Amit", "marks": 70 }
```

```
{ "name": "Priya", "marks": 95 }
```

```
{ "name": "Kiran", "marks": 60 }
```

Combined Example

```
db.students.aggregate([  
  { $match: { marks: { $gt: 70 } } },  
  { $project: { name: 1, marks: 1, _id: 0 } },  
  { $sort: { marks: -1 } },  
  { $limit: 3 }  
)
```

Viva Questions:

1. What is aggregation in MongoDB?

Aggregation in MongoDB is a process of analyzing and transforming data in a collection to produce summarized or computed results.

2. What is an aggregation pipeline?

An aggregation pipeline is a framework where documents pass through multiple processing stages, with each stage performing a specific operation on the data.

3. Which stage is used for grouping documents?

The **\$group** stage is used to group documents based on a specified field.

4. What does the `_id` field represent in the `$group` stage?

The `_id` field specifies the key by which documents are grouped, such as department name.

5. What is the use of `$sum: 1` in aggregation?

\$sum: 1 increments the count by 1 for each document, effectively counting the number of documents in each group.

Conclusion:

In this experiment, the MongoDB aggregation pipeline was successfully used to analyze student data. By applying the \$group stage, students were grouped according to their departments, and the total count of students in each department was calculated. This experiment demonstrates how aggregation helps convert raw data into meaningful insights, making MongoDB suitable for analytical and reporting applications.